Swing-Trajectory Residual Reinforcement Learning for Rough-Terrain Quadruped Locomotion

Jaechan Lee
Dept. of Robotics and Mechatronics Engineering
DGIST
Daegu, South Korea

Dohoon Kim
Dept. of Robotics and Mechatronics Engineering
DGIST
Daegu, South Korea

Beomjun Jeong
School of Undergraduate Studies
DGIST
Daegu, South Korea

Abstract—Model-based quadruped controllers built on Single-Rigid-Body (SRB) model predictive control (MPC) and whole-body control (WBC) rely on a nominal foot-swing plan whose timing and clearance are fixed by hand-tuned gait parameters. On rough terrain, an unexpected terrain height or contact timing error can turn this nominal swing into a hard touchdown, disturbing body pitch and roll and degrading the following steps. This project implements a residual reinforcement learning (RL) interface in which the policy sends a planning-level swing-trajectory residual into the C++ controller. The 12-dimensional action has three channels per leg: Bezier mid-curve x-shape bias, swing-duration delta, and swing-height delta. PolicyCore clamps and scales this action, forwards it to PlanningCore, and PlanningCore applies it to LegFSM swing timing and FootSwingTrajectory Bezier control points while keeping the nominal touchdown endpoint fixed. Zero residual behavior is designed to be identical to the baseline controller, while nonzero actions alter only the foot-swing trajectory path before the command reaches the existing SRB-MPC/WBC control stack. On randomized rough terrain the MPC-only baseline fails to sustain locomotion, whereas the same MPC controller augmented with the learned swing residual keeps its balance and traverses the terrain.

Keywords—residual reinforcement learning; quadruped locomotion; swing trajectory; Bezier curve; model predictive control; rough terrain

I. Project Overview

Rough-terrain quadruped locomotion frequently violates the assumptions behind nominal gait and foot-swing planners. The baseline used in this project combines SRB-MPC with a WBC-style leg controller. SRB-MPC stabilizes the body through planned ground reaction forces, but the foot swing trajectory that delivers the next contact is generated from fixed Bezier waypoints, fixed swing duration, and fixed toe clearance. When the terrain height or contact timing differs from the nominal plan, the robot can meet the ground too early, too late, or with insufficient clearance.

The implemented core idea is a swing-trajectory residual: a low-dimensional RL policy adjusts the shape, duration, and clearance of each swing step while preserving the existing MPC/WBC controller. The residual does not replace MPC torques. Instead, it augments the PlanningCore inputs that define the next foot-swing trajectory, so the downstream controller tracks a slightly adapted reference.

The platform is the MCL Quad, a quadruped with biarticular closed-chain legs. The residual channel acts on planning-level swing parameters because these are the quantities actually consumed by the deployed PolicyCore and PlanningCore path; the active residual is [Bezier shape, swing dt, swing height] per leg.

The project contributes the following:

- A swing-trajectory residual RL structure that preserves the existing SRB-MPC/WBC controller and modulates only PlanningCore foot-swing parameters.
- A 12-D action contract with safe scaling and clamping: Bezier x-shape delta, swing-duration delta, and swing-height delta for each of four legs.
- A C++ PolicyCore/PlanningCore injection path and Python binding aliases that expose the same residual to MuJoCo playback and Isaac Lab MPC-residual environments.

Fig. 1. Quadruped robot.

II. Activities Performed

The project ran over roughly twelve days (May 19–May 31, 2026). The three team members—Jaechan Lee, Dohoon Kim, and Beomjun Jeong—carried out all tasks jointly; weekly activities are summarized in Table I.

TABLE I. Table I. Weekly Activities (Carried Out Jointly)

Week / Period	Activities (joint)
W1 (05-19)	Problem scoping and design of the swing-trajectory residual interface; decision to keep the residual at the planning level, decoupled from direct torque or gain commands.
W2 (05-20–05-22)	PolicyCore C++ residual-injection module; pybind11 binding exposure; migration of the residual action semantics to swing-trajectory parameters.
W3 (05-23–05-25)	PlanningCore integration: leg-wise swing duration, Bezier shape, and swing-height residuals; logging/debug channels for residual deltas and phase diagnostics.
W4 (05-26–05-28)	Isaac Lab MPC-residual environment work, PPO configuration, conservative residual

	scales, and MuJoCo playback of the C++ MPC with residual actions.
W5 (05-29–05-31)	Rough-terrain training to convergence, baseline comparison of MPC-only versus MPC-plus-residual locomotion, and final report consolidation.

III. Methodology

A. System Overview

The system has two practical paths. In MuJoCo and the C++ controller, PolicyCore receives the latest 12-D action through set_swing_residual, clamps it to [-1, 1], scales it, and forwards a FootSwingPolicyCommand to PlanningCore. In Isaac Lab, the MPC-residual environment uses the C++ MPC bridge as the base controller and treats the RL action term as a raw 12-D value holder. In both cases, the residual changes foot-swing planning parameters; the existing MPC/WBC stack still produces body stabilization and final joint torques.

B. Residual Action Space

The policy action has three channels per leg—[Bezier x-shape, Δswing_dt, Δswing_height]—for a total of 12 dimensions, each bounded to [-1, 1]. The C++ default scales are 0.15 stride-ratio units, 0.04 s, and 0.06 m, while the Isaac Lab MPC-residual configuration uses more conservative shakedown scales. After scaling, PlanningCore applies additional safety clamps before using the command.

C. Phase Diagnostics

PolicyCore still computes a touchdown-traversal gate from the previous PlanningOutputPacket: a late-swing ramp and an early-stance ramp-down. In the current swing-trajectory design, this gate is diagnostic and observation-oriented; it is not multiplied into the trajectory command. PlanningCore needs the raw residual at swing start so that the next Bezier curve and swing duration are selected consistently.

D. Planning Application

PlanningCore applies the residual in three places. First, the Bezier shape channel shifts the airborne x control points stride_ratio[0..2], while the touchdown endpoint stride_ratio[3] stays fixed so the nominal landing point remains governed by stride and capture-point correction. Second, the swing_dt channel changes base_T_phase into an effective leg-wise swing duration, which also changes the LegFSM swing ratio and stride calculation. Third, the swing_height channel shifts the airborne z control points z_coord[0] and z_coord[1], while touchdown z targets remain nominal. The average effective swing ratio is published for scalar consumers such as the HMM contact estimator.

E. Reward Design

The reward and evaluation targets are aligned with rough-terrain swing robustness:

- commanded linear / angular velocity tracking;
- foot clearance, stumble, and premature-contact penalties;
- post-touchdown body roll/pitch angular-velocity penalty;
- excessive vertical body-velocity, terrain collision, and torque-saturation penalties;
- residual action magnitude, action-rate, and smoothness regularization.

F. Asymmetric Training and Deployment

The MPC-residual path exposes the deployed C++ swing-trajectory residual directly through the pybind11 Batch-Controller. MuJoCo provides a closed-chain verification path for the C++ MPC plus swing residual, while Isaac Lab provides the GPU-parallel rough-terrain training and integration target where the baseline and residual controllers are compared.

G. Evaluation Protocol

The proposed swing residual is compared against two baselines, all evaluated on the same terrain where available (Table II). The key check is whether learned swing timing, shape, and clearance reduce hard touchdown events and improve survival compared with fixed or analytic swing settings.

TABLE II. TABLE II. EVALUATION BASELINES

Baseline	Description
(a) MPC/WBC only	Nominal Bezier swing trajectory, fixed swing duration and height, no residual.
(b) MPC + analytic swing schedule	Hand-tuned phase or terrain-aware swing-height/duration schedule, no learning.
(c) MPC + learned swing residual	Proposed method.

Metrics: touchdown GRF peak and impulse, premature-contact or stumble frequency, foot-clearance violations, post-touchdown body roll/pitch disturbance, velocity-tracking error, survival rate and traversed distance over rough terrain, and torque-saturation frequency.

IV. Execution Result

A. PolicyCore C++ Residual Injection

The PolicyCore module and its pybind11 binding inject a 12-D swing-trajectory residual (per leg [Bezier shape, Δswing_dt, Δswing_height]) into the C++ MPC/WBC controller. The injection path is PolicyCore::SetResidualAction → PolicyCore::runFromPacket → PlanningCore-InputPacket.foot_swing_policy → PlanningCore. The zero-residual case is intended to be bit-identical to residual-off behavior, because PolicyCore returns zero commands when disabled and PlanningCore receives zero deltas. A nonzero residual changes the Bezier control points, effective swing duration, and toe clearance, confirming that the action affects the planning path as intended.

B. Isaac Lab Residual Environment and Training

The Isaac Lab MPC-residual environment was implemented around the C++ MPC bridge. The action manager supplies a 12-D residual, the environment clamps and reorders it per leg, and set_swing_residual_batch forwards it to the C++ BatchController. Residual debug buffers mirror phase_norm, gate_w, sched_contact, and the three residual delta groups for observations, logging, and reward terms. PPO configuration uses gentler exploration

scales because swing trajectory residuals directly perturb timing and clearance.

C. Rough-Terrain Baseline Comparison

On the randomized rough-terrain course, the MPC-only baseline (a) fails to sustain locomotion. With its swing duration, Bezier shape, and toe clearance fixed by the nominal gait, the leg meets the uneven ground too early or with insufficient clearance, producing hard and premature touchdowns. The resulting roll and pitch disturbance accumulates over successive steps, and within a few strides the body loses balance and the robot falls. Under the same terrain and the same velocity commands, the MPC controller augmented with the learned swing residual (c) keeps its balance and walks across the course. By adapting swing duration, Bezier shape, and toe clearance step by step, the policy softens touchdown impact, restores foot clearance over obstacles, and suppresses the post-touchdown attitude disturbance that destabilizes the baseline. The learned swing residual is therefore the deciding factor between failure and successful traversal on rough terrain, while it leaves the MPC torques and the nominal touchdown endpoint unchanged.

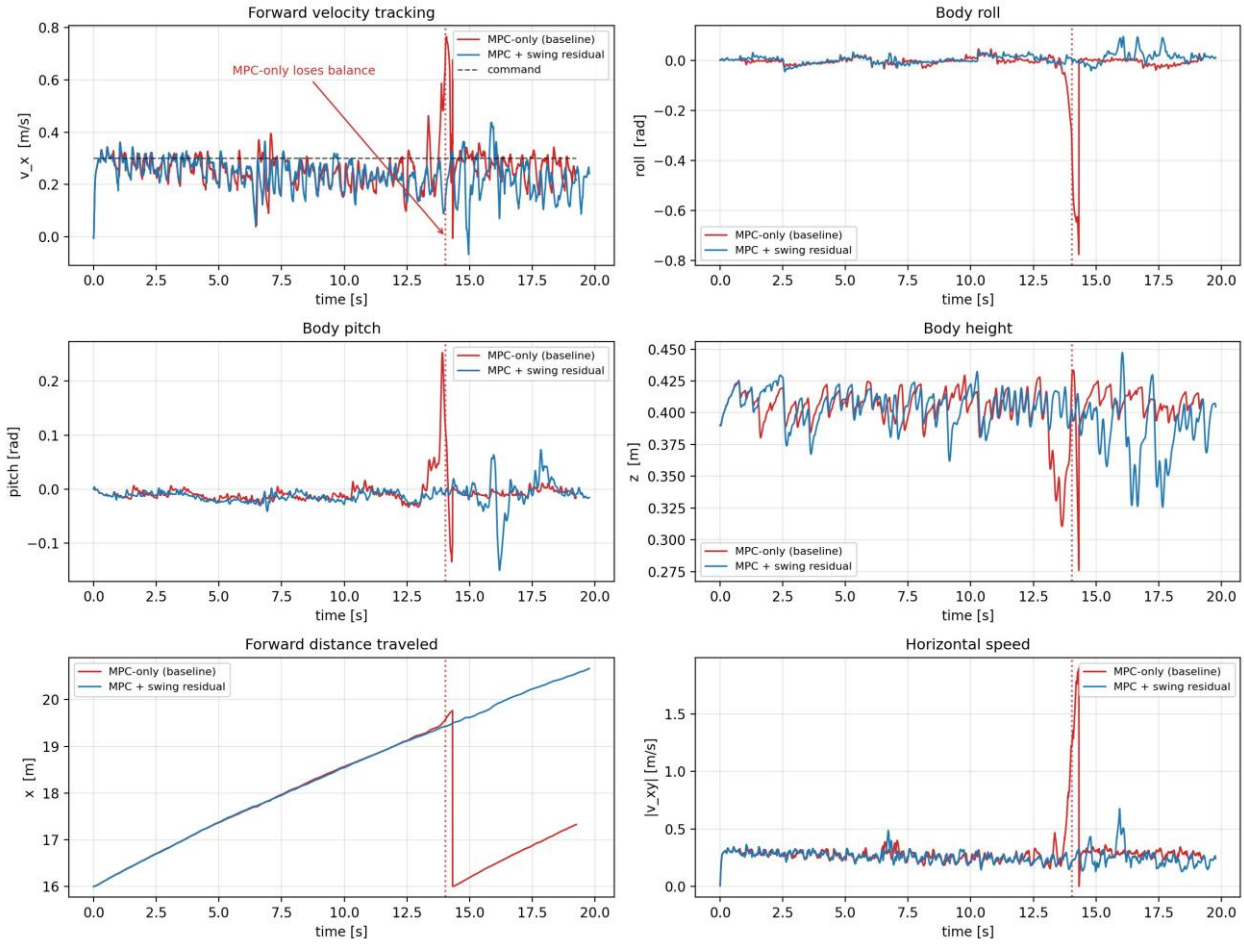

Fig. 2. MPC-only baseline versus MPC + swing residual on rough-terrain play. From top-left: forward velocity tracking, body roll, body pitch, body height, forward distance traveled, and horizontal speed. The MPC-only baseline loses balance near t ≈ 14 s, while the MPC + swing residual controller keeps tracking the command and continues to traverse the terrain.

V. Result Analysis

The rough-terrain comparison shows that the fixed swing plan is the limiting factor for the model-based controller. The MPC keeps the body upright on flat ground, but on uneven terrain the nominal swing timing and clearance are no longer appropriate, and the controller cannot recover from the repeated hard touchdowns they cause. Allowing the policy to adjust only the swing reference—while leaving the MPC torques and the touchdown endpoint untouched—is enough to turn the failing baseline into a controller that traverses the terrain, which confirms that planning-level swing adaptation, rather than a change of controller, is what rough-terrain locomotion requires here.

The PolicyCore/PlanningCore path resolves the main integration risk of the current residual interface. Zero action preserves the nominal planner, while nonzero action changes only leg-wise swing references before the controller tracks them. The diagnostic gate remains useful for observation and analysis, but the trajectory residual itself is raw and planning-level so that swing-start decisions are coherent.

Limitations and next steps. The present result establishes the qualitative outcome—MPC-only fails while MPC-plus-residual succeeds on rough terrain—but a full quantitative sweep is still in progress. The remaining work is to report touchdown impact, foot-clearance violations, attitude disturbance, velocity-tracking error, and survival rate as numbers with statistics over many terrain seeds, to compare the learned residual against a hand-tuned analytic swing schedule (b), and to extend the evaluation to a wider range of terrain roughness and locomotion speeds.

VI. References

[1] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2018, pp. 1–9.

[2] T. Johannink et al., "Residual reinforcement learning for robot control," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2019, pp. 6023–6029.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[4] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in Proc. Conf. Robot Learn. (CoRL), 2021, pp. 91–100.

[5] M. Mittal et al., "Orbit: A unified simulation framework for interactive robot learning environments," IEEE Robot. Autom. Lett., vol. 8, no. 6, pp. 3740–3747, 2023.

[6] M. H. Raibert, Legged Robots That Balance. Cambridge, MA, USA: MIT Press, 1986.

[7] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2012, pp. 5026–5033.

[8] C. D. Bellicoso, F. Jenelten, C. Gehring, and M. Hutter, "Dynamic locomotion through online nonlinear motion optimization for quadrupedal robots," IEEE Robot. Autom. Lett., vol. 3, no. 3, pp. 2261–2268, 2018.